

Complejidad algorítmica y asintótica

Cuando creamos un algoritmo nos tenemos que preguntar si resuelve el problema como queremos que lo haga. Si es incorrecto no resolverá el problema, pero si es correcto puede hacerlo de manera ineficiente. Por ejemplo, puede ser demasiado caro y por lo tanto inaceptable.

Decimos que es caro respecto a los recursos que utiliza. Los recursos que nos interesan son el espacio de memoria necesaria y el tiempo de ejecución.

La memoria se mide por la cantidad de variables que utiliza **y ¿el tiempo de ejecución?** En esta materia nos ocuparemos de este recurso de ahora en adelante.

La medida del tiempo de ejecución se tomará en función de la cantidad de instrucciones que ejecuta el algoritmo.

Complejidad algorítmica

Nos da una medida del esfuerzo que hay que realizar para correr un algoritmo y lo costoso que este proceso resulta.

La eficiencia suele medirse en términos de consumo de recursos:

Espaciales: cantidad de memoria que un algoritmo consume o utiliza durante su ejecución →

Complejidad espacial

Temporales: tiempo que necesita el algoritmo para ejecutarse → **Complejidad temporal**

Nos centraremos en **complejidad temporal** por ser el recurso más crítico

Para continuar con el estudio de la complejidad temporal tenemos que hablar de otros conceptos:

- El estudio asintótico del problema
- Las cotas de una función

¿De una función? ¿No hablábamos de algoritmos? ... en un rato lo verán más claro

Suponemos que vamos a estudiar algoritmos que... **la cantidad de veces que se ejecutan sus instrucciones dependerá del tamaño de su entrada... ¿?**

- ¿A qué nos referimos con la entrada del algoritmo?
- ¿y qué significa su tamaño?
- ¿Hay algoritmos cuyo tiempo de ejecución no dependen de la entrada?

Complejidad Asintótica

Consiste en el cálculo de la complejidad temporal de un algoritmo en función del tamaño del problema, n , prescindiendo de factores constantes multiplicativos y suponiendo valores de n muy grandes.

No sirve para establecer el tiempo exacto de ejecución, sino que permite especificar una cota (inferior, superior o ambas) para el tiempo de ejecución de un algoritmo

- Es **asintótico** porque nos interesa estudiar la complejidad de un algoritmo cuando el tamaño de la entrada es muy grande.
- La **cota** que buscamos nos dirá, en el peor caso, el peor comportamiento que puede tener nuestro algoritmo en estudio. Para prevenir las consecuencias de su ejecución, especialmente cuando el tiempo de ejecución sea crítico.

A continuación, compararemos gráficamente las principales funciones que representan la complejidad de los algoritmos.

En estos gráficos, n es el tamaño de la entrada. Si la entrada es un arreglo de 100 elementos, entonces $n=100$.

El eje de la variable dependiente $t(n)$ representa la cantidad de instrucciones que debe ejecutar el algoritmo en función del tamaño de la entrada n , que es la variable independiente en el eje X.

Es importante comparar el ritmo de crecimiento de las funciones del Gráfico 1 con las del Gráfico 2 y pensar ejemplos que permitan entenderlo.

Gráfico de complejidad asintótica (cuando n tiende a infinito), para las funciones más conocidas:

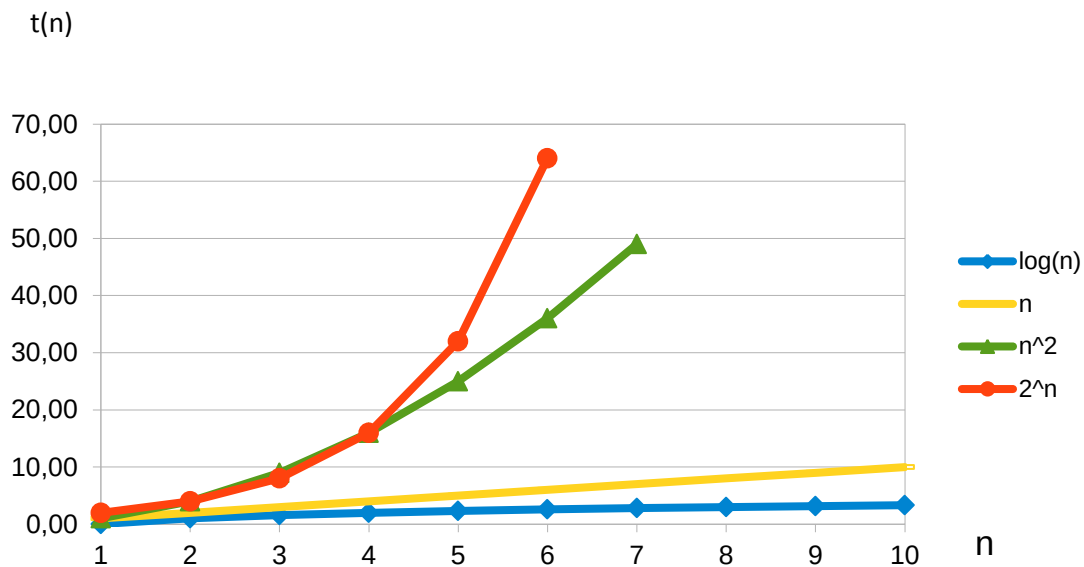


Gráfico1: Familias de funciones

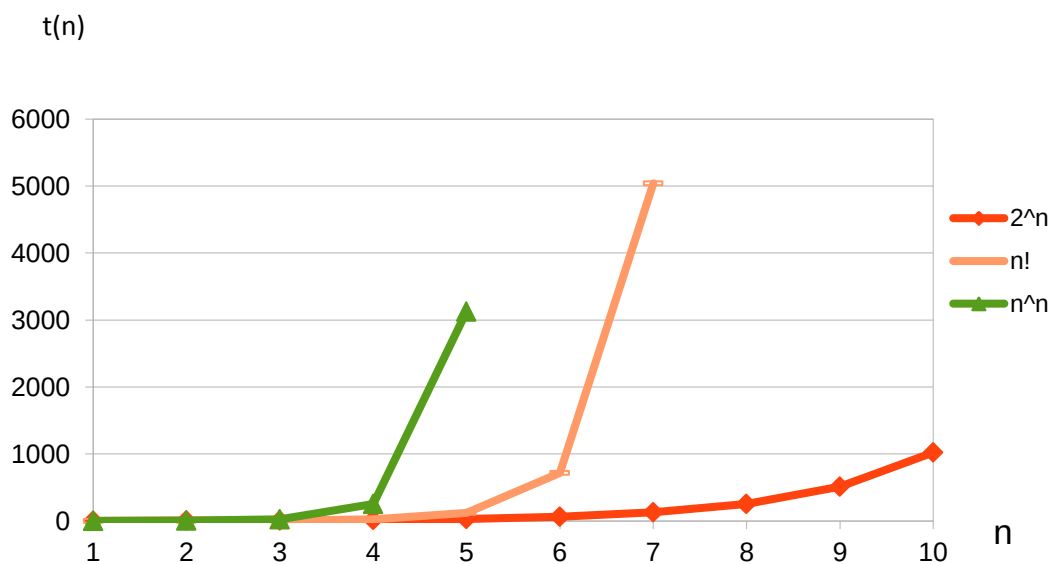


Gráfico2: Familias de funciones

Se puede utilizar la complejidad algorítmica para comparar dos implementaciones del mismo algoritmo y quedarnos con la implementación que tenga mejor complejidad asintótica.

Ejemplo de complejidad asintótica

Ejemplo en pseudocódigo:

Buscar el máximo valor de un array

maximoArray(A) {	
maxActual = A [0];	operaciones 2
PARA i = 1 HASTA n – 1 HACER {	1 + 2 n
SI (A[i] > maxActual)	2(n – 1)
ENTONCES maxActual=A[i];	2(n – 1)
{ incrementar contador i }	2(n – 1)
}	
Return maxActual;	1
}	
	Total 8n – 2

El orden asintótico de maximoArray es:

$$t(n) = 8n - 2$$

Las cotas

Una cota se define como un número M tal que cualquier elemento del conjunto es menor o igual que M.

Ejemplo1

Para el conjunto $M = \{2, 5, 6\}$ podemos dar varias cotas:

$$C_1 = 6$$

$$C_2 = 7$$

Notar que todo el conjunto es “menor o igual a 6” pero también es “menor o igual a 7”, y “menor o igual a 8” y “menor o igual a 50”. Por lo tanto: 6, 7, 8 y 50 son cotas válidas.

En el caso de una función, $f(n)$, una cota está dada por otra función a la que llamaremos $g(n)$, tal que para todo n :

$$g(n) > f(n)$$

Ejemplo2:

Como se ve en el Gráfico1, podemos acotar $f(n) = \log(n)$ por $g(n) = n$, porque para todo $n \in \text{Nat}$:

$$\text{Cota1: } n > \log(n)$$

Pero también podríamos acotar $\log(n)$ por $g(n) = n^2$

$$\text{Cota2: } n^2 > \log(n)$$

Siempre que sea posible, vamos a dar la cota más ajustada, en este caso, Cota1.

En el ejemplo 1, la cota más ajustada es 6.

Estas definiciones nos sirven para definir cómo vamos a dar una medida asintótica de la complejidad algorítmica:

La notación O

Una definición formal de la complejidad es la siguiente:

$$f \in O(g) \Leftrightarrow \exists n_0, c \text{ tales que para todo } n > n_0 \\ f(n) < c g(n)$$

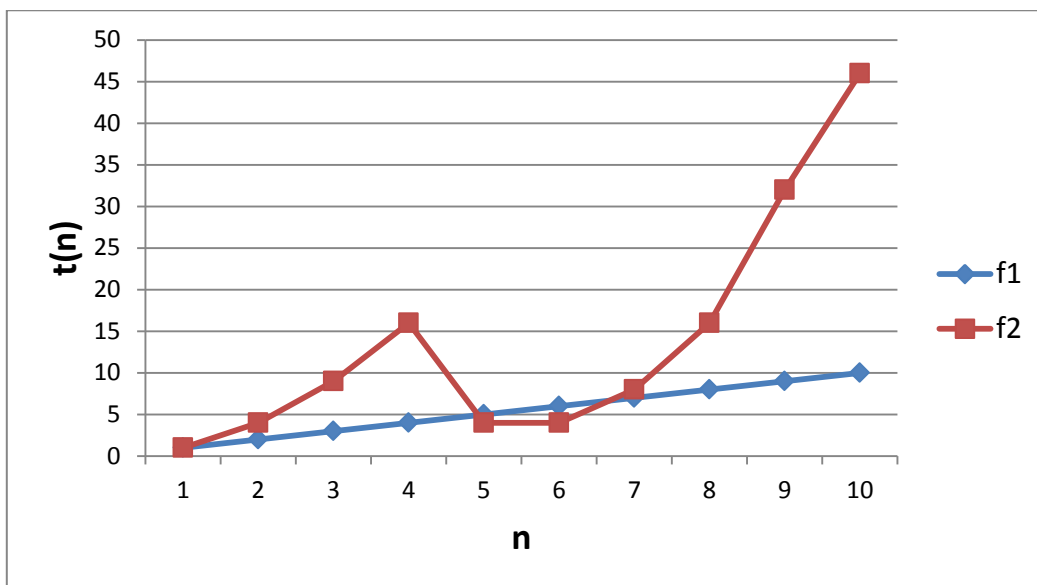
- Donde **n**, es el tamaño de la instancia con la que se ejecuta el algoritmo,
- **c** es una constante multiplicativa que permite determinar a g como cota de f
- y **n_0** significa, "el primer valor de n " para el cual se cumple que g es cota de f

Veamos cómo podemos utilizar esta definición para estudiar la complejidad computacional:

Ejemplo1:

Queremos probar que $f1 \in O(f2)$

- Es decir, que cuando n tiende a infinito $f2 > f1$
- Es decir, queremos **acotar** $f1$ con $f2$.



1) Buscamos n_0

El primer candidato es 2, ya que $f_2(2) > f_1(2)$. Sin embargo, $f_2(6) < f_1(6)$.

Entonces, no es cierto que "para todo $n > 2 \rightarrow f_2(n) > f_1(n)$ ".

✓ Si $n_0 = 7$, sí podemos afirmar que "para todo $n > 7 \rightarrow f_2(n) > f_1(n)$ ".

Siempre que f_1 y f_2 crezcan a la velocidad que sugiere el gráfico.

2) En este caso podemos usar $c = 1$ como constante multiplicativa. Se entenderá al relacionar con otro ejemplo en el que c sea distinto de 1.

**Ejemplo2:**

Ahora vamos a acotar *máximoArray* utilizando la definición de O

- Sea $f(n) = 8n - 2$

Esto lo sabemos porque ya contamos las operaciones a mano.

- Sea $g(n) = n$

$g(n)$ en este punto es una conjetura.

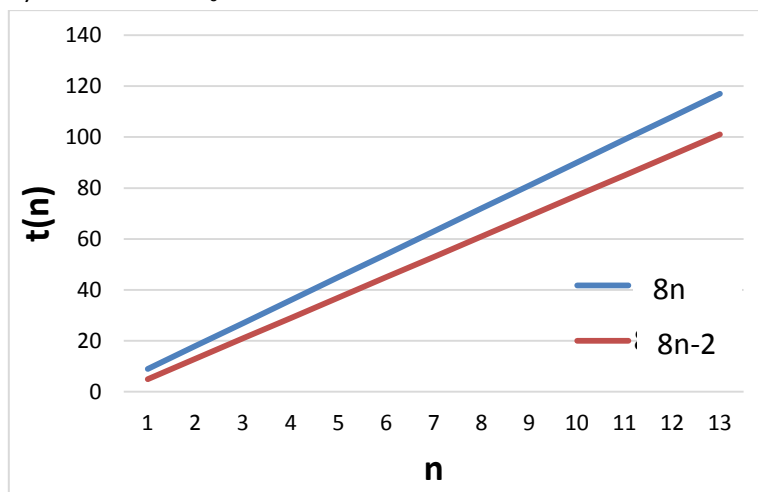
Si queremos demostrar que $8n-2 \in O(n)$ entonces tenemos que encontrar un c y un n_0 tal que:

$$8n-2 < c \cdot n \quad // \quad f(n) < c \cdot g(n)$$

para todo $n > n_0$

1) Claramente $c > 7$, porque de lo contrario, no es cierto que $8n-2 < 7 \cdot n$. Tomaremos $c=8$

2) Buscamos el n_0



El candidato es $n_0 = 1$.

Como f y g son lineales, sabemos que crecen a la misma velocidad y que por lo tanto no van a volver a cruzarse.

Por lo tanto, para todo $n > 1 \rightarrow 8n > 8n-2$

Con (1) y (2), demostramos que $8n-2 \in O(n)$

Donde $g(n) = 8n$ es una cota de $f(n) = 8n-2$

Con este análisis llegamos a la conclusión:

La función de complejidad del algoritmo está acotada por una función lineal, $g(n)$, y podemos afirmar que es $O(n)$.

Veremos sobre esta notación a continuación.

Existen diferentes notaciones para la complejidad asintótica

Una de ellas es la notación O , que permite especificar la cota superior de la ejecución de un algoritmo.

La sentencia " $f(n)$ es $O(g(n))$ " significa que la tasa de crecimiento de $f(n)$
no supera la tasa de crecimiento de $g(n)$

La notación " O " sirve para clasificar las funciones de acuerdo con su tasa de crecimiento e intuitivamente nos permite agrupar las funciones "parecidas" o sea, del mismo tipo.

Lineales:	$f(x) = a \cdot x + b$	$\Rightarrow f \in O(x)$
Cuadráticas:	$f(x) = a \cdot x^2 + b \cdot x + c$	$\Rightarrow f \in O(x^2)$
Cúbicas:	$f(x) = a \cdot x^3 + b \cdot x^2 + c \cdot x + d$	$\Rightarrow f \in O(x^3)$
Constantes:	$f(x) = k$	$\Rightarrow f \in O(1)$
Exponenciales:	$f(x) = a \cdot b^{c \cdot x} + d$	$\Rightarrow f \in O(b^x)$
Logarítmicas:	$f(x) = a \cdot \log(x) + b$	$\Rightarrow f \in O(\log(x))$
Factorial:	$f(n) = a \cdot n! + b$	$\Rightarrow f \in O(n!)$

Material Programación I

Importante: no son conjunto disjuntos, o sea

$$O(n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \dots$$

Seguimos recordando lo aprendido en Programación I:

- Intuitivamente, $f \in O(g)$ si $\alpha \cdot g(n)$ “le gana” a $f(n)$ para valores grandes de n , para algún α cualquiera fijo.
- Ejemplos:
 - Si $f(n) = n$ y $g(n) = n^2$, entonces... $f \in O(g)$.
 - Si $f(n) = n^2$ y $g(n) = n$, entonces... $f \notin O(g)$.
 - Si $f(n) = 100n$ y $g(n) = n^2$, entonces... $f \in O(g)$.
 - Si $f(n) = 4n^2$ y $g(n) = 2n^2$, entonces... $f \in O(g)$ (y a la inversa).
- Como dijimos, la idea es agrupar funciones “parecidas” ...

En resumen podemos decir que la notación O proporciona una cota superior para la tasa de crecimiento de una función:

Dadas $f(n)$ y $g(n)$:	$f(n)$ es $O(g(n))$	$g(n)$ es $O(f(n))$
$g(n)$ crece más	Sí	No
$f(n)$ crece más	No	Sí
Igual crecimiento	Sí	Sí

Propiedades de $O(f(n))$:

Reflexiva: $f(n) \in O(f(n))$

Transitiva: si $f(n) \in O(g(n))$ y $g(n) \in O(h(n))$, entonces $f(n) \in O(h(n))$

Eliminación de constantes: $O(c \cdot f(n)) = O(f(n))$, para todo c .
 $O(\log_a n) = O(\log_b n) = O(\log n)$, para todo a y b .

Suma de órdenes: $O(f(n) + g(n)) = O(\max(f(n), g(n)))$

Producto de órdenes: $O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$

¿Es simétrica?

Álgebra de Ordenes

Reglas y Álgebra de Ordenes

- 1) $O(1) \leq O(\log n) \leq O(n) \leq O(n^k) \leq O(k^n) \leq O(n!) \leq O(n^n)$
- 2) $O(f) + O(g) = O(f + g) = O(\max\{f, g\})$
- 3) $O(f) \cdot O(g) = O(f \cdot g)$
- 4) $O(k) = O(1)$ para todo k constante.

Ejemplo: $O(2n)$
 $= O(2) \cdot O(n)$ // por regla 3
 $= O(1) \cdot O(n)$ // por regla 4
 $= O(n)$

➤ Resolver para $O(n \cdot \log n) + O(n)$

- 5) $\sum_{i=1}^k O(f) = O(\sum_{i=1}^k f) = O(k \cdot f)$
 Si k es constante, entonces vale $O(f)$

$$6) \sum_{i=1}^k i = \frac{k \cdot (k+1)}{2} = O(k^2)$$

El análisis asintótico de algoritmos determina el tiempo de ejecución
utilizando la notación “O”

Para realizar el análisis asintótico

- Buscar el número de operaciones primitivas ejecutadas en el peor de los casos como una función del tamaño de la entrada
- Expresar esta función con la notación “O”

Ejemplo:

- Se sabe que el algoritmo *maximoArray* ejecuta como mucho $8n-2$ operaciones primitivas
- Se dice que *maximoArray* “se ejecuta en un tiempo $O(n)$ ”

Como se prescinde de los factores constantes y de los términos de orden menor, se puede hacer caso omiso de ellos al contar las operaciones primitivas

Análisis del caso mejor, peor y medio

El tiempo de ejecución de un algoritmo puede variar con los datos de entrada.

Ejemplo (ordenación por inserción):

```

INICIO
i ← 1
MIENTRAS (i < n)
    x ← Arreglo[i]
    j ← i - 1
    MIENTRAS (j > 0 Y ARREGLO[j] > x)
        ARREGLO[j+1] ← ARREGLO[j]
        j ← j - 1
    FIN-MIENTRAS
    ARREGLO[j+1] ← x
    i ← i + 1
FIN-MIENTRAS
FIN
    
```

- 1) Si el vector ya está completamente ordenado de forma creciente (mejor caso) → el segundo bucle no realiza ninguna iteración → Complejidad $O(n)$.
- 2) Si el vector está ordenado de forma decreciente (el peor caso) entonces se cumple que x es menor que $ARREGLO[j]$ para todo j entre 1 e $i-1$. La salida del bucle se produce cuando $j = 0$.
 - i. En este caso, el algoritmo realiza $i - 1$ comparaciones.
 - ii. Esto será cierto para todo valor de i entre 1 y $n - 1$ si el orden es decreciente.
 - iii. El número total de comparaciones será:

$$\sum_{i=1}^{n-1} (i - 1) = \frac{n \cdot (n - 1)}{2} \in O(n^2)$$

- 3) En el caso promedio o sea, cuando está en cualquier orden diferente a los casos 1) y 2), la complejidad estará entre $O(n)$ y $O(n^2)$.
 - Para hallarla habría que calcular matemáticamente la complejidad de todas las permutaciones de todos valores que se pueden almacenar en el vector.
 - **Se puede demostrar que es $O(n^2)$.**

Ejercicio1

Demostrar que $\sum_{i=0}^n i = (n+1) * n / 2$

Veamos el caso particular:



$$\sum_{i=0}^{10} i = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55$$

Si agrupamos los números de a pares, como se muestra:

$$10 + 1; 9 + 2; 8 + 3; 7 + 4; 6 + 5$$

Vemos que hay 5 grupos que suman 11 $\rightarrow \Rightarrow 10/2 * 11 = 55$

Para cualquier n tenemos

$$1 + 2 + 3 + \dots + n-3 + n-2 + n-1 + n$$

Se puede observar que se tienen **$n/2$** grupos y que todos suman **$n+1$** , la suma total es **$(n+1) * n/2$**

Y demostramos lo pedido.

Continuando con el análisis, podemos ver qué orden de complejidad le corresponde a esta función cuando es la función que se obtiene al contar la cantidad de operaciones de un algoritmo:

- $\frac{1}{2}n^2 + \frac{1}{2}n < 2n^2$
- $2n^2$ está en $O(2) O(n^2)$
- $O(2) O(n^2)$ **está en $O(n^2)$**

Ejercicio2 Combinatoria

En una habitación hay **n** personas que se quieren saludar entre sí.

Las reglas para saludarse son:

- 1) Cada persona puede saludar solo una persona a la vez.
- 2) El saludo es simétrico: Si A saluda a B, se considera que B saluda a A en el mismo saludo.
- 3) Cada saludo demora 1 segundo

4) Si hay n personas, puede haber hasta $n/2$ saludos simultáneos.

Se pide analizar y responder:

a) ¿Cuánto tiempo (en segundos) se necesita para que todas las personas queden saludadas?

Ayuda: Hacer una tabla para el caso de 4 y de 8 personas.

b) ¿Cuántos saludos hay en total?

c) ¿Cómo cambia a) si cambiamos la regla 4) de manera que solo puede haber un saludo a la vez?

d) ¿Cómo cambia a) si cambiamos la regla 1) de manera que cada persona puede saludar a más de una persona a la vez?

e) ¿Cómo cambia a) si quitamos la regla 1) y además, cada persona que es saludada sale de la habitación?

Ayuda: Hacer una tabla para el caso de 4 y de 8 personas.

	1	2	3	4
1	x	1	3	2
2		x	2	3
3			x	1
4				x

Tabla1: Tabla para 4 personas

Como se ve en la tabla1, para 4 personas se necesitan 3 segundos.

La notación O en dos variables

Ejemplo

Sea f un algoritmo trivial que suma dos vectores $v1$ y $v2$

- ¿Cuáles son las variables que definen el tamaño de la entrada?

Claramente la complejidad no puede depender solo del tamaño de $v1$ o del tamaño de $v2$.

Es necesario contemplar una cota que dependa de ambas situaciones:

- Sea $n1$ el tamaño de $v1$
- Sea $n2$ el tamaño de $v2$

Se puede dar una definición formal de la complejidad para dos variables:

$$f_{n,k} \in O(g_{n,k}) \Leftrightarrow \exists n_0, k_0, c \text{ tales que para todo } n > n_0 \text{ y para todo } k > k_0$$

$$f_{(n,k)} < c g_{(n,k)}$$

- Donde n y k son el tamaño de la instancia que es parámetro del algoritmo.
- y n_0 significa, "el primer valor de n " para el cual se cumple.

Ejercicio3

Demostrar por definición que la complejidad de f está en $O(n + k)$

ANEXO

- Tiempos empleados para el cálculo de algoritmos con distintos ordenes, considerando que el computador en cuestión ejecuta 1 Millón de operaciones por segundo (1MHz).

$\begin{matrix} T(n) \\ n \end{matrix}$	$\log n$	n	$n \log n$	n^2	n^3	2^n	$n!$
10	$3.3 \cdot 10^{-6}$	10^{-5}	$3.3 \cdot 10^{-5}$	10^{-4}	0.001	0.001	3.63
50	$5.6 \cdot 10^{-6}$	$5 \cdot 10^{-5}$	$2.8 \cdot 10^{-4}$	0.0025	0.125	intratable	intratable
100	$6.6 \cdot 10^{-6}$	10^{-4}	$6.6 \cdot 10^{-4}$	0.01	1	intratable	intratable
10^3	10^{-5}	0.001	0.01	1	1000	intratable	intratable
10^4	$1.3 \cdot 10^{-5}$	0.01	0.13	100	10^6	intratable	intratable
10^5	$1.6 \cdot 10^{-5}$	0.1	1.6	10^4	intratable	intratable	intratable
10^6	$2 \cdot 10^{-5}$	1	19.9	10^6	intratable	intratable	intratable